
실습(5): 병렬 합

윤 승 현

동국대학교 컴퓨터·AI학부

커널1(많은 분기)

```
__global__ void parallel_sum1(float* dev_in, float* dev_out, int size)
{
    __shared__ float partialSum[NumThread];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // 현재 블록의 데이터를 공유 메모리로 복사
    partialSum[tid] = (i < size) ? dev_in[i] : 0.0f;
    __syncthreads();

    // 분기가 많은 병렬 합 수행
    for (int s = 1; s < blockDim.x; s *= 2) {
        if ((tid % (2 * s)) == 0)
            partialSum[tid] += partialSum[tid + s];
        __syncthreads();
    }

    // 현재 블록의 결과를 전역 메모리에 기록
    if (tid == 0)
        dev_out[blockIdx.x] = partialSum[0];
}
```

커널2(적은 분기)

```
__global__ void parallel_sum2(float* dev_in, float* dev_out, int size)
{
    __shared__ float partialSum[NumThread];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // 현재 블록의 데이터를 공유 메모리로 복사
    partialSum[tid] = (i < size) ? dev_in[i] : 0.0f;
    __syncthreads();

    // 분기가 적은 병렬 합 수행
    for (int s = blockDim.x / 2; s > 0; s /= 2) {
        if (tid < s)
            partialSum[tid] += partialSum[tid + s];
        __syncthreads();
    }

    // 현재 블록의 결과를 전역 메모리에 기록
    if (tid == 0)
        dev_out[blockIdx.x] = partialSum[0];
}
```

병렬 합(Parallel Sum)

```
#define Size (1024 * 1024 * 512)          // 2GB
#define NumThread 512

int main()
{
    // CPU 메모리 할당 및 초기화
    float* M = new float[Size];
    for (int i = 0; i < Size; i++)
        M[i] = rand() / (float)RAND_MAX;    // 0.0 ~ 1.0
    printf("초기화 완료...\n");

    cudaSetDevice(0);

    // GPU 메모리 할당 및 복사
    float* dev_M1, *dev_M2;
    cudaMalloc((void**)&dev_M1, Size * sizeof(float));
    cudaMalloc((void**)&dev_M2, Size * sizeof(float));
    cudaMemcpy(dev_M1, M, Size * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(dev_M2, M, Size * sizeof(float), cudaMemcpyHostToDevice);
}
```

병렬 합(Parallel Sum)

```
// 1. CPU 순차 합
auto st = std::chrono::high_resolution_clock::now();
double cpu_sum = 0.0;
for (int i = 0; i < Size; ++i)
    cpu_sum += M[i];
auto ed = std::chrono::high_resolution_clock::now();
double cpu_ms = std::chrono::duration<double, std::milli>(ed - st).count();

printf("CPU 경과시간 = %lld ms\n", (long long)cpu_ms);
printf("순차 합 = %.31f\n\n", cpu_sum);

// 2. GPU 병렬 합(많은 분기)
st = std::chrono::high_resolution_clock::now();
double gpu1_sum = gpu_reduce_sum(dev_M1, Size, false);
ed = std::chrono::high_resolution_clock::now();
double gpu1_ms = std::chrono::duration<double, std::milli>(ed - st).count();

printf("GPU 경과시간(많은 분기) = %lld ms\n", (long long)gpu1_ms);
printf("병렬 합(많은 분기) = %.31f\n\n", gpu1_sum);
```

병렬 합(Parallel Sum)

```
// 3. GPU 병렬 합(적은 분기)
st = std::chrono::high_resolution_clock::now();
double gpu2_sum = gpu_reduce_sum(dev_M2, Size, true);
ed = std::chrono::high_resolution_clock::now();
double gpu2_ms = std::chrono::duration<double, std::milli>(ed - st).count();

printf("GPU 경과시간(적은 분기) = %lld ms\n", (long long)gpu2_ms);
printf("병렬 합(적은 분기) = %.3lf\n\n", gpu2_sum);

// 메모리 해제
cudaFree(dev_M1);
cudaFree(dev_M2);
delete[] M;
cudaDeviceReset();

return 0;
}
```

병렬 합(Parallel Sum)

```
double gpu_reduce_sum(float* dev_src, int size, bool less_branch)
{
    float* dev_in = dev_src;
    float* dev_out = nullptr;    // 블록을 부분합을 저장할 배열
    int curSize = size;

    // 커널 반복 호출
    while (true) {
        int gridSize = (curSize - 1) / NumThread + 1;    // 블록의 수
        cudaMalloc((void**)&dev_out, gridSize * sizeof(float));    // 블록의 수 만큼 배열 할당

        dim3 dimGrid(gridSize, 1);
        dim3 dimBlock(NumThread, 1, 1);
        // 나머지 부분을 완성 하시오.
        // .....
    }
    float result = 0.0f;
    cudaMemcpy(&result, dev_out, sizeof(float), cudaMemcpyDeviceToHost);
    cudaFree(dev_out);
    return (double)result;    // 결과 반환
}
```

수행 결과

- 결과 비교

배열 크기	CPU 순차 합	GPU 병렬 합(많은 분기)	GPU 병렬 합(적은 분기)
1024 * 1024 * 128			
1024 * 1024 * 256			
1024 * 1024 * 512			
1024 * 1024 * 1024			

Q&A